Ingresos API

API sob alta demanda - projeto individual

Nome do aluno: João Pedro Wanderley do Nascimento
Repositório GitHub: https://github.com/KonradKurzeVIII/ProjetoHugoIngressos
Ensaio principal: 11/11/2026 às 19:39 (America/Sao_Paulo).

1. Tecnologias e organização

Node.js 24, TypeScript, Fastify 5 e PostgreSQL 18. O serviço HTTP usa JWT HS256; senhas usam scrypt assíncrono e salt aleatório. O banco guarda usuários, show e pedidos. A interface usa HTML, CSS e JavaScript locais, servidos pela API. Docker Compose reúne aplicação e banco, com volume persistente, healthchecks e porta 8080 publicada somente no localhost.

src/app.ts concentra rotas, autenticação e validação. **src/orders.ts** concentra estados e cotas; **src/fifo.ts** ordena mutações. **src/database.ts** controla transações e conexões; **db/schema.sql** cria o schema e o índice parcial das reservas. A pasta public contém a interface; test contém integração HTTP/PostgreSQL e prova da fila.

2. Endpoints e comportamento implementado

POST /auth/registrar: recebe username/password; responde 201. **POST /auth/login**: responde 200 com token. Todas as rotas seguintes exigem Authorization: Bearer <token>.

POST /pedidos: aceita showId 7; cria NO_CARRINHO e responde 201. **POST /pedidos/{id}/reservar**: responde 200 com RESERVADO e **X-Lote: 7F3A**; recusas usam 409 com POOL_CHEIO ou COTA_PESSOAL. A chave do corpo é sempre **error**, com a grafia exata do contrato.

POST /pedidos/{id}/cancelar: responde 200 com CANCELADO e libera a vaga e a cota pessoal. **GET /pool**: retorna total, ocupadas e livres. Pedido alheio ou inexistente responde 404 nas duas mutações. O endpoint adicional público **GET /health** verifica acesso ao banco.

Aplicação web em http://localhost:8080: cadastro, login, carrinho, reserva, cancelamento com confirmação e consulta da disponibilidade. O endpoint adicional autenticado **GET /pedidos** lista somente os pedidos da conta. Recarregar a página ou entrar novamente recupera o estado do banco, inclusive cancelamentos.

Concorrência e garantias

3. Estratégia escolhida

Uma única instância admite reservas e cancelamentos em uma **fila FIFO comum**, depois de autenticar e validar a requisição. Chegada significa essa admissão no servidor, não o horário do clique nem a chegada de pacotes. A requisição aguardando na fila não mantém conexão com o banco nem transação aberta.

Quando chega sua vez, a operação abre uma transação READ COMMITTED e executa **SELECT ... FOR NO KEY UPDATE** no show 7. Sob esse bloqueio, verifica proprietário, estado, quantidade global e quantidade pessoal. Só então atualiza o pedido. Confirma com COMMIT antes de responder; uma falha executa ROLLBACK. A conexão é liberada mesmo se houver erro.

Reservas e cancelamentos sempre obtêm o mesmo bloqueio primeiro. Portanto, duas operações não podem decidir ao mesmo tempo com base na mesma vaga livre. A fila fornece ordem; o bloqueio e a transação fornecem atomicidade. Um lock de banco, isoladamente, não promete FIFO. O modo NO KEY UPDATE permite criar carrinhos sem esperar pela reserva, pois é compatível com o bloqueio da verificação de chave estrangeira.

4. Invariantes e liberação

O número de pedidos RESERVADO permanece entre **0 e 100**; cada usuário mantém **no máximo dois**. Carrinhos não ocupam vaga. A ocupação é derivada dos pedidos, sem contador de estoque duplicado. GET /pool usa uma única consulta e snapshot MVCC; ocupadas + livres é sempre 100.

Repetir uma reserva confirmada retorna 200 e X-Lote sem ocupar outra vaga. Repetir um cancelamento confirmado retorna 200 sem liberar outra vaga. Somente RESERVADO pode mudar para CANCELADO. Cancelamento não apaga o registro e não permite reservar novamente esse mesmo pedido.

5. Decisões complementares e espera

O projeto tem apenas o show 7. Outros showId recebem 400. Estado incompatível recebe 409 com ESTADO_INVALIDO. Propriedade vem antes de estado e cotas. Para uma nova reserva, COTA_PESSOAL precede POOL_CHEIO quando ambos se aplicam. Credenciais e JWT inválidos recebem 401.

A API desativa deadlines internos de processamento. O pool tem dez conexões, sem expirar a espera; statement_timeout e lock_timeout são zero. As transações são curtas, sem chamadas externas. Headers incompletos e sockets ociosos seguem os limites normais do Node. Falhas reais de infraestrutura não são disfarçadas como falta de ingresso.

Testes e estado da entrega

6. Como a aplicação foi testada

Carga e testes de contrato usam PostgreSQL real e conexões HTTP locais, sem injeção HTTP do Fastify. A suíte valida autenticação, propriedade, estados, header, limite pessoal, retries concorrentes, cancelamentos, rollback e limites globais com filas independentes. Também consulta o banco para verificar as invariantes. Os 14 testes automatizados passaram e incluem assets web, isolamento da listagem, criação de carrinho sem esperar pelo bloqueio e recuperação da fila após uma desconexão sem código do driver pg injetada de forma controlada.

No ensaio principal em containers Linux, 1.000 usuários e carrinhos foram preparados por SQL; JWTs válidos foram assinados pela aplicação. Mil conexões HTTP foram estabelecidas gradualmente antes de disparar as reservas juntas. A medição exclui cadastro, login e abertura de conexão. Esse desenho mede requisições concorrentes, sem incluir o custo de abertura das conexões TCP.

1000 tentativas simultâneas: 100 sucessos; 900 recusas POOL_CHEIO; 0 respostas inesperadas; 0 timeouts.
Latência: p50 757,03 ms; p95 1.247,08 ms; p99 1.282,28 ms.
Vazão: 750,13 requisições/s. **Pool:** 1570 leituras consistentes. FIFO de admissão e estado final no banco foram verificados.

Uma segunda carga externa cadastrou e autenticou usuários pelo HTTP e executou 400 reservas simultâneas contra o executável compilado: 100 sucessos, 300 recusas, zero resposta inesperada e timeout. Ao final, cancelou suas reservas e confirmou 100 vagas livres.

Verificação no navegador: cadastro, login com senha inválida, novo login, duas reservas, recusa da terceira mantendo o carrinho, confirmação de cancelamento e recuperação dos pedidos após recarregar. O layout foi inspecionado em desktop e nas larguras 390 e 320 pixels, sem rolagem horizontal.

7. Reproduzir e apresentar

Entrega: docker compose down -v; depois docker compose up --build -d. Acessar http://localhost:8080. O primeiro comando apaga os dados do volume.

Testes: usar docker-compose.yml junto de docker-compose.test.yml e executar o serviço tests, conforme README. Para desenvolvimento com Node 24: npm ci, npm run typecheck, npm run build e npm test. O roteiro de cinco minutos está em docs/apresentacao.md.

8. Limitações e pendências conhecidas

FIFO global depende de uma instância. A fila pendente reside em memória e não sobrevive à queda da API; commits persistem e retries são seguros. A latência cresce com a fila. A evidência é limitada à carga executada: não garante ausência de falhas do sistema operacional ou recursos infinitos.

Docker Compose validado. Build, inicialização pelos dois comandos da avaliação, healthchecks, 14 testes e carga HTTP na API de produção passaram no GitHub Actions, em Ubuntu 24.04. Evidência do commit **00575fdde934**: execução de conformidade. Os relatórios e a referência estão em output/. O envio ao classroom é feito pelo aluno.